

Quiz #5 Answer Sheet

CS 225 Data Structures

Spring 2026

7:00-8:00pm, Thursday, April 23

Name	Hanbin Zheng
ZJU ID	
Lab TA	

- This is a **closed book** and **closed notes** exam. No electronic aids are allowed, either.
- You should have 4 problems total on 4 pages. You must write on the exam.
- Unless otherwise stated in a problem, assume the best possible design of a particular implementation is being used.
- Unless the problem specifically says otherwise, (1) assume the code compiles, and thus any compiler error is an exam typo (though hopefully there are not any typos), and (2) assume you are NOT allowed to write any helper methods to help solve the problem, nor are you allowed to use additional arrays, lists, or other collection data structures unless we have said you can.
- Please put your name at the top of each page.
- Please fill in your **Lab Section** or your **Lab TA's name** to help us organize and grade your papers.
- Don't panic and good luck!

Problem 1	8	/10
Problem 2	7	/10
Problem 3	8	/10
Problem 4	8	/10

+3

Total	30	/40
-------	----	-----

33



Name:

2

Problem 1 answer sheet

1.

Maximum number of children per node: m

Maximum number of keys per node: $m - 1$

Minimum number of children (non-root): $\lceil \frac{m}{2} \rceil$

Minimum number of keys (non-root): $\lceil \frac{m}{2} \rceil - 1$

2.

a) 5 children

b) No

3.

$$(m-1)(1+m+m^2+\dots+m^h) = \frac{m^{h+1}-1}{m-1}(m-1) = m^{h+1}-1 \quad m=3, h=2$$
$$3^{(2+1)}-1 = 3^3-1 = 26$$

4.

Root: ~~(6, 1)~~

Left child of root: ~~(5, 4)~~

Right child of root: (8, 7)

Left child of (8, 7): ~~(7, 2)~~

(6, 1) $\rightarrow$ (8, 7) $\rightarrow$ (7, 2) $\rightarrow$ _____

5.

~~A~~



Problem 2 answer sheet

1. This will introduce lots of collision.

2.

[0] NUL [1] 12 [2] 23 [3] 34 [4] 45 [5] 56
[6] 67 [7] 78 [8] NUL [9] NUL [10] NUL

[0] NUL [1] 12 [2] 34 [3] NUL [4] 67 [5] 45
[6] 23 [7] 78 [8] 56 [9] NUL [10] NUL

3.

Successful find: $O(1 + \frac{\alpha}{2})$ $1 + \frac{\alpha}{2}$
Unsuccessful find: $O(1 + \alpha)$ $1 + \alpha$

4. rehashing will resize the hash table (to nearest 2-time prime) and add old keys to new hash table (insert one by one).

Problem3:

1. b

3. ~~c~~ cd

2. bde

4. cd



Problem 4:

```

template <class K, class V>
void BTree<K, V>::insert(BTreeNode* subroot, const DataPair& pair)
{
    if(subroot == nullptr) return;
    size_t first_larger_idx = insertion_idx(subroot->elements, pair);
    // TODO: Your code goes here
    if (!subroot->is_leaf) { 檢查 first_larger_idx
        insert(subroot->children[first_larger_idx], pair); 插入
    2 } else {
        subroot->elements.insert(subroot->elements.begin() + first_larger_idx, pair);
        return;
    }
    if (subroot->children[first_larger_idx]->elements.size() >= order)
    {
        split_child(subroot, first_larger_idx);
    }
}

```

